Web & Mobile Developer

Ju-cheol Park

Full-stack AI document analysis and settlement platforms, with web applications and Android integration for business workflows.

Experience

Triphos Co., Ltd. / 2026.02 ~ 2026.11

Scope

Built web interfaces and server APIs, and integrated Android app features

jukrap628@gmail.com github.com/jukrap www.linkedin.com/in/jukrap/ jukrap.vercel.app/en

Selected work

01 AI-Powered Document and Data Analysis Platform

Fixed the target statements on the server for Ragas evaluation and matched the returned evaluation results by statement ID. Batched long inputs and checked quotations, supporting evidence, and answer coverage separately.

02 Transport Settlement and Vehicle Management ERP and Mobile App

Rejected stale saves while retaining drafts. Users compare original, current and edited values before confirming a new save.

03 Contract-Customer Parcel Web and Shipping Label App

Reduced the initial JavaScript file size by about 74% by loading screens and Excel code on demand. Used device logs and physical printouts to fix printer connection failures and clipped labels.

04 AI Agent Playbook

A personal tool for project rules and skills, with installation recovery and protection for user-edited files.

Company work 01

AI-Powered Document and Data Analysis Platform

Migrated an existing Python system to Django and React, preserving screen, API and data contracts while improving retrieval, answer validation and Ragas evaluation.

Period
2026.09 ~ 2026.10

Scope
Led Django and React migration and feature integration

Technologies React, TypeScript, Tailwind CSS, Vite, TanStack Query, Zustand, Python, Django, MariaDB, SQLGlot, Gemma 4, Ragas, tokenizers, Vitest

Problem

Migrated an existing Python system to Django and React while checking screen, API and data parity. Subsequent work addressed long answers exceeding evaluation limits and evaluators changing the meaning of target statements.

Fix the evaluation targets and validate the verdicts

Fixed the target statements on the server for Ragas evaluation and matched the returned evaluation results by statement ID. Batched long inputs and checked quotations, supporting evidence, and answer coverage separately.

Verification

Completed evaluation of 25 claims extracted from an answer about a real document in five batches. Checks reject missing or duplicate IDs and invalid response types before malformed verdicts can be stored as valid results.

Separate model latency from application overhead

Profiled internal processing with fixed model responses, then removed repeated token counting and reused document pages already processed within the request.

Verification

After warmup, 20 runs before and after the change showed median internal processing fall from 2,969ms to 672ms. Model call counts stayed unchanged.

Outcome

Long-answer evaluations can finish, and repeated computation causes less internal delay. Answers appear first, followed by asynchronous Ragas evaluation, so users do not wait for evaluation to finish.

Median internal application processing time

2,969ms → 672ms

Compared application processing with fixed model responses: 20 runs per condition after warm-up, excluding model and network wait time.

Completed evaluation of a long answer within the input limit

25 items evaluated

Evaluated a Gemma 4 answer about a real document in 5 batches. The largest request used 8,164 tokens, below the 8,192-token input limit.

No need to wait for quality evaluation

Display the answer first, then evaluate it separately

Users can read the answer before Ragas finishes. The next evaluation request is deferred while a new user question is being processed.

Company work 02

Transport Settlement and Vehicle Management ERP

Built the frontend and extended an existing backend for vehicle, driver, billing and settlement workflows, preserving input during conflicts and protecting finalized records.

Period
2026.07 ~ 2026.10

Scope
Led frontend development and backend API expansion

Codebase size
Approx. 256,000 lines across frontend and backend
Source total: comments, blank lines, etc. excluded

Technologies React, TypeScript, Tailwind CSS, Vite, React Router, Zustand, Spring Boot, MyBatis, MariaDB, Vitest

Problem

Concurrent edits must preserve both another operator's saved changes and the user's unsaved input.

Block stale saves and let users resolve the differences

Rejected stale saves while retaining drafts. Users compare original, current and edited values before confirming a new save.

Verification

Reproduced conflicts in development. Checked draft and database preservation after 409 responses, then compared resolved saves with the database and change history.

Preserve Excel validation through the save step

Rechecked server-held validation data, permissions and references before saving. Failed batches roll back together; retries return their existing result.

Verification

Reproduced intermediate failures, retries, and concurrent requests in an isolated MariaDB environment to check for partial writes and duplicate application of the same request.

Outcome

Preserved finalized amounts separately from current assignments. Documented screen, API and database connections and environment-specific launch settings for handover.

Company work 02 / Mobile

Transport Settlement Companion App

Developed screens and server APIs for the ERP-connected mobile app. Connected React business screens and Spring Boot APIs to notifications and file downloads in Flutter.

Period
2026.07 ~ 2026.10

Scope
Full-stack development · excluding the chat system

Technologies React, TypeScript, TanStack Query, TanStack Router, Spring Boot, MariaDB, Flutter, Dart, Firebase Cloud Messaging

Problem

The transport settlement app needed to retain the latest state despite delayed or repeated notifications. On cold start, navigation targets could arrive before the web interface was ready.

Separate notification content from read receipts

Version and deletion checks prevent a late creation event from reviving a deleted notification. Read receipts are stored separately; a single UPSERT replaces an update-then-insert sequence while preserving the first seen time.

Verification

Reproduced a receipt deadlock in isolated MariaDB, then reran concurrent requests. Reversed and repeated notification events to verify that the latest content and receipt state were retained.

Open the intended work screen after a cold start

Separated WebView loading from React listener readiness. Retained notification events in Native code until the web readiness signal arrived, and reset readiness on navigation.

Verification

Verified notification navigation in a test APK after dismissing the app from recents. Also checked delivery before web readiness and after navigation to a new page.

Outcome

Connected inquiry handling, notifications and PDF downloads to the ERP, including current notification state and download access checks in the mobile app.

Company work 03 / Web

Contract-Customer Parcel Booking and Delivery Management Web App

Built the complete frontend for a logistics web app covering search, reservations, Excel imports and label printing. Integrated server APIs and PC printers, and connected the same business screens to label printing in an Android app.

Period
2026.04 ~ 2026.06

Scope
Complete frontend development

Technologies React, TypeScript, Vite, React Router, TanStack Query, Zustand, XLSX, Vitest

Problem

The operations web app loaded screens and Excel code that were unnecessary at startup. I separated code needed on entry from code needed when a user starts a feature to reduce the initial download size.

Use bundle analysis to find and split unnecessary startup code

Used bundle analysis to identify code that was not needed on entry. Loaded page code when users entered each screen and Excel libraries when they started an upload or download.

Verification

In a standard production build, initial JavaScript fell from 2,405.50kB to 616.60kB, or 815.10kB to 204.38kB with gzip. Checked that navigation, data queries, Excel operations, and PC label printing still worked after the change.

Check physical output separately from the preview

Did not treat the on-screen layout as proof of print quality. Wrapped long addresses to the available width and adjusted type size and spacing.

Verification

Printed physical labels to find and correct clipped addresses and overlapping fields.

Outcome

Reduced the code required at entry and refined the path from selected operational data to physical labels.

Initial JavaScript file size

2,405.50 → 616.60 kB

Initial JavaScript file size reduced by about 74%

Gzip-compressed size

815.10 → 204.38 kB

About 75% smaller for the compressed initial JavaScript file

Company work 03 / Android

Contract-Customer Mobile Shipping Label App

Connected web label-print requests to native modules and Bluetooth printers in an Android app. Built device selection, connection checks and the print-request flow.

Period
2026.04 ~ 2026.06

Scope
Android app development

Technologies Expo, React Native, Expo Router, WebView, Android native module, TypeScript, Bluetooth

Problem

Android could select a printer but still failed to connect. An SDK acknowledgment also could not establish that a physical label had finished printing.

Trace the connection failure into the SDK

Device logs showed that the SDK stopped device discovery during connection. Because that operation also requires scan permission, the app now checks both scan and connection permissions.

Verification

Checked the permission prompt, printer connection, and physical label output on an Android 16 device.

Show when a print request has been sent

Displayed “Print request sent” when the SDK accepted the request.

Verification

Checked SDK responses and printed labels separately to align the wording with what the app can actually observe.

Outcome

Connected React Native screens to the Android printer SDK and handled permission and connection failures separately.

Label-printing path

01
WebView
Label data and print request

02
Android
Bridge, permissions, device connection

03
Bluetooth
Printer SDK and label output

Project 01

AI Agent Playbook

Built a tool that helps AI coding tools refer to project rules and work history across sessions. Users can select skills to install, change the setup and recover it while preserving existing files.

Period
2026.06 ~ Present

Role
Solo developer

Technologies JavaScript, Node.js, MCP SDK, Zod, GitHub Actions

Problem

I wanted project decisions and work status to carry over when starting a new AI session or changing coding tools.

Connect project records and skills through CLI / MCP

Provided access to project rules and records through a CLI and read-only MCP. Package installation, project initialization, skill installation and MCP setup can be applied independently.

Verification

Checked that the CLI and MCP operate on the same project records and that installation and initialization paths work independently, with CI verification on Windows and Ubuntu.

Installation and recovery that preserve user edits

Used ownership metadata and hashes to detect modified files. Validated all new skills before cleaning up existing files, rechecked changes immediately before deletion, and retained backups and an application log for recovery.

Verification

Tested partial installation failures, file conflicts, changes made after inspection, and restoration. Checked that a failed installation does not remove existing files or user edits.

Outcome

Published the project record structure, CLI and MCP server as an npm package with public source and usage documentation.

GitHub
github.com/jukrap/ai-agent-playbook

npm
www.npmjs.com/package/ai-agent-playbook

Company work 04

Stock Auto-Investment and Account Management Web App

Built the complete React frontend for account, stock and order administration using MSW mock APIs. Shared table components and query-state handling across recurring screens.

Period
2026.02 ~ 2026.03

Scope
Complete frontend development

Technologies React, TypeScript, Vite, React Router, TanStack Query, Zustand, MSW

Shared tables, filters, and modals

Built shared list, search, detail and editing components for account, stock and order screens. Added column pinning, resizing and reordering to tables, and implemented queries and input interactions with MSW mock APIs.

Applying order-history filters

Kept account and stock selections separate from the conditions applied to the list. Editing a filter retained the existing results; pressing Search applied the submitted conditions to the TanStack Query key.

Repeat searches and load more

Search again with the same filters

Reset the existing query to reload from the first page.

Guard load-more requests

Skipped additional requests when no next page remained or a request was already in progress.

Company work 05

Location-Based Daily Information App

Added public-data weather features to an existing Android app, implementing the WebView interface, server-side API integration, and caching.

Period
2026.06 ~ 2026.07

Scope
Frontend and backend feature development

Technologies Spring MVC, JSP, jQuery, Java, Android

Weather data across separate APIs

Weather, air quality, and alerts had different regional coverage and update schedules. Fetching everything on every visit repeated requests for the same area and could hold up available information while a slower response arrived.

Caching by region and data type

Reuse data for requested regions

Cached responses by region and data type, with expiry periods matched to each type.

Share in-flight requests

Reused an in-flight result for the same cache key rather than issuing another external API call.

Show core information first

Displayed current conditions and the core forecast first, then filled in air quality and alerts as responses arrived. Delays or failures in supplementary requests did not hide weather data already received.

Handling stale information

When an external request failed, allowed stale values could remain visible. An expired “no alerts” response was marked unavailable rather than treated as current. Previously retrieved active alerts were labelled as based on the last retrieval.

Company work 06

Phishing Prevention and Credit Information Protection App

Added email-breach lookup and deepfake image checks to an existing Android app. Connected inspection screens, server calls to external APIs, and Android image selection from input through results.

Period
2026.06

Scope
Frontend and backend feature development

Technologies Android, Cordova, Spring MVC, jQuery, Java

Turn external inspection APIs into app features

Implemented server calls to external APIs for submitted emails and images, translating responses and errors for the interface. API credentials remained on the server; the app received the results and status needed for display.

From image selection to inspection

Input validation and preview

Validated the selected image's file type, size and resolution, and displayed a preview before submission.

Android file selection

Opened the Android file picker for image or file requests from the WebView and returned the selected file to the WebView.

When the user selects another image

A previous file read can finish after the user selects another image. Assigned a token to each read and checked it on completion so stale results would not overwrite the new preview.

Company work 07

Project Analysis and Development Documentation Assistant

Built an internal web tool for drafting and reviewing documents from repositories and project kickoff material. Implemented screens and server APIs for collecting sources, reviewing requirements, editing tables and exporting documents.

Period
2026.04

Scope
Frontend and backend development

Technologies Node.js, TypeScript, React, AI API, xlsx, Vitest

Review source material before drafting

Split the interface into review stages for proposed requirements and open questions before drafting final documents. Enabled functional specifications after the requirements document was ready, and screen specifications after their prerequisite documents were ready.

Review tables and revise selected content

Edit document sheets

Made requirements, functional specifications and screen specifications editable as separate sheets. Exported documents from the tables as edited by the user.

Targeted AI revisions

Sent the selected sheets or cells together with surrounding content. Distinguished the revision target from content to preserve so users could change the selected part without regenerating the entire document.

Handle partial generation failures

Tracked generation status per document so completed documents remained available for review after another failed. Supported regenerating and exporting selected documents without restarting the entire set.

Company work

Further company work

Feature additions to existing services and updates for Android versions and field use.

08 School Transport and Attendance Management Platform

Period
2026.09

Platform
Web / Android

Implementation

Added member search, editing, duplicate merging, inquiries and attachments to existing web and Android apps. Checked member roles and authorship on the server and connected file selection and image previews in the app.

Technologies Spring MVC, Java, MyBatis, Vue 2, MariaDB, Android, Kotlin

09 Parcel Pickup and Delivery Operations Platform

Period
2026.08

Platform
Web / Android

Implementation

Updated map-marker selection and return-to-map behavior. Retained delivery-completion requests and photos offline, restored requests after app restarts and compared them with completed server records before retrying.

Technologies React, TypeScript, React Native, Expo, Spring Boot, SQLite

10 Chart Editor

Period
2026.03 ~ 2026.04

Platform
Web

Implementation

Built chart settings, data connections and previews for a website under development. Connected chart-specific data fields and reduced unnecessary recreation to limit flicker and scroll movement.

Technologies React, TypeScript, Vite, Chart.js, WebGL/GLSL (OpenGL-style), OGL (WebGL library), MSW, Vitest

11 Warehouse Operations and Scanning App

Period
2026.03 ~ 2026.04

Platform
Android

Implementation

Added receiving filters, quantity entry and QR/barcode scanning to an existing PDA app. Separated validation, duplicate handling and quantity rules by work mode, and used device input and logs to distinguish external validation failures from app errors.

Technologies Android Java, Gradle/AGP, Scanner SDK

12 Mobile Inventory and Shipping App

Period
2026.03

Platform
Android

Implementation

Aligned compatible development-tool versions to restore builds of a legacy Android app. Updated OS-specific permissions, file access and back navigation and checked behavior on devices running older OS versions.

Technologies Android Java, Gradle/AGP, RxJava, FileProvider

Project 02

C. Donghae

Built a map-based web service for Donghae Line passengers to find live transport and nearby information.

Period

2024.10.04 ~ 2024.10.06

Role

Frontend developer / team of three

Technologies TypeScript, React, Next.js, Tailwind CSS, Google Maps, Storybook, AWS, GitHub Actions, Docker

Hackathon scope

The team had 72 hours to combine train, weather, and local-place data in one map with mobile interactions.

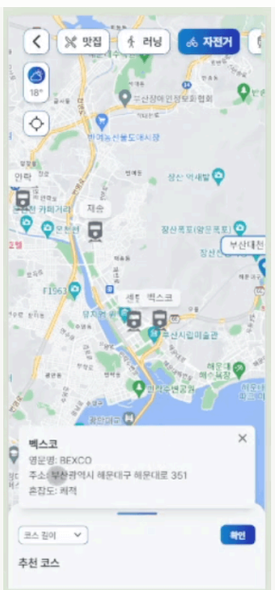
Implementation

Maps and API integration

Worked with two backend developers using Swagger documentation to connect APIs and build the main Google Maps screens.

Mobile map interaction

Built a bottom sheet so users could browse places while keeping the map visible. The sheet expands or collapses according to drag distance and velocity.





Award

Won the Busan Technopark President Award at DIVE 2024, placing third in the challenge track.

Project 03

ShareBBy

Developed Android support and community features for an app that helps people share hobbies and find companions.

Period

2024.04 ~ 2024.06

Role

React Native app development and Firebase integration

Technologies JavaScript, React Native, Firebase, Faster Image

Android support and image updates

Screens initially built for iOS needed Android support, and stale image caches prevented updated post images from appearing.

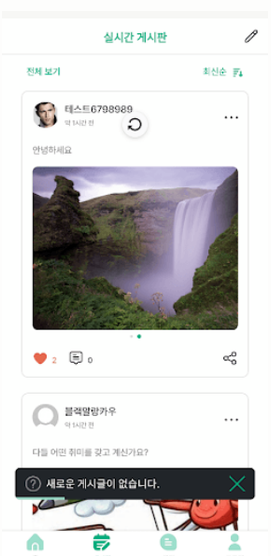
Implementation

Community features

Built post and comment creation, reading, editing and deletion, with location filters and sorting. Users can pull to refresh the list or scroll to load more posts.

Data and image handling

Documented Firebase relationships in an ERD, reproduced the caching problem, and adopted a maintained replacement library.





Release

Delivered Android screens and community features. The team released the app on the App Store in 2024.

GitHub

github.com/jukrap/archived-rn-ShareBBy

Project 04

Itzip

Led frontend work in a 15-person project for aspiring developers. Built blog and Markdown editing screens and configured tests, error monitoring and deployment infrastructure.

Period

2024.07 ~ 2025.06

Role

Frontend team lead / DevOps

Technologies TypeScript, React, Next.js, Tailwind CSS, React Spring, Jotai, Jest, Prisma, Storybook, Postman, Sentry, AWS, Jenkins, Docker

Editing and team development

The team needed a shared way to inspect component behavior and errors after deployment while building the editor.

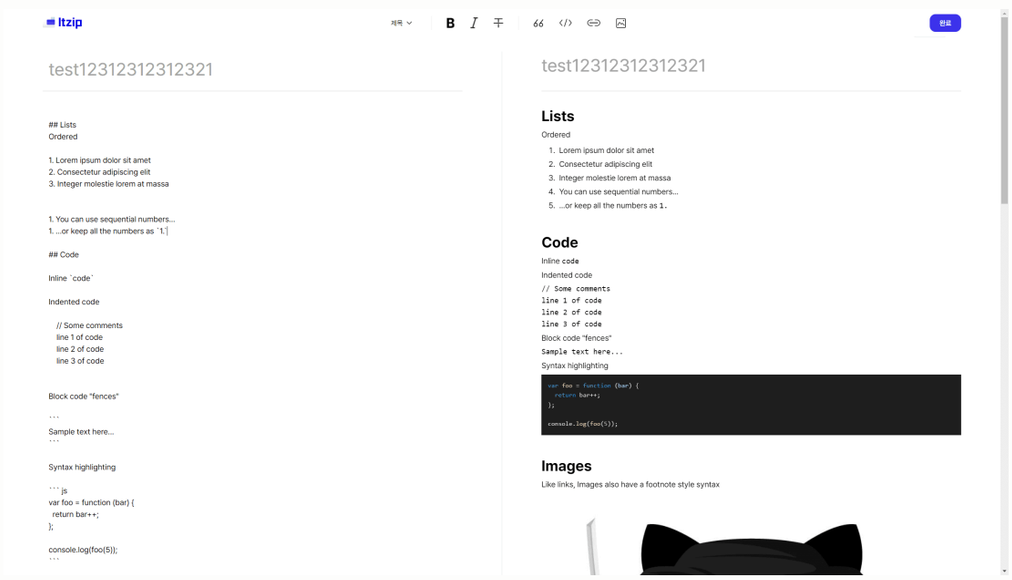
Implementation

Markdown editing and previews

Built a Markdown editor with live previews so writers could see the result as they typed. Supported project-specific syntax and loaded heavy post components on demand through dynamic imports.

Tests and error monitoring

Tested key behavior with Jest, documented components in Storybook, and configured Sentry for post-deployment errors.



Markdown editing and preview

Team adoption

Delivered blog and editor screens with shared tests, component documentation and error monitoring. Also configured the AWS runtime and deployment infrastructure with Jenkins and Docker.

GitHub

github.com/ITZipProject/itzip_front

Project 05

Posture Teacher

An Android app that finds body landmarks in camera frames, classifies sitting and plank posture, and records how long each posture is held.

Period

2022.07 ~ 2023.05

Role

Android developer / team of two

Technologies Java, Android, Jetpack, MediaPipe, SQLite

Camera analysis and interaction

Continuous frame analysis needed to remain separate from screen input, and MediaPipe had to be built for the Android project.

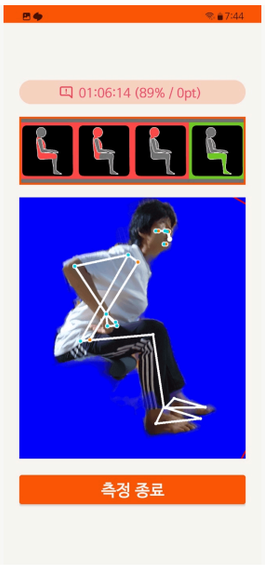
Implementation

MediaPipe integration and posture detection

Built a MediaPipe AAR on Ubuntu and used landmark angles and lengths to classify posture and measure hold times.

Separate frame analysis

Moved analysis to a separate thread so camera processing and screen input did not block each other.



Posture analysis and timing records

Measured result

During the project, camera frame-processing throughput (FPS) with MediaPipe was 5–10 times that of the earlier OpenCV implementation.

GitHub

github.com/jukrap/archived-Posture-Teacher

Profile

Skills, education, and awards

Skills

Languages

TypeScript, JavaScript, Java, Python, Dart

Web

React, Next.js, Vite, TanStack Query, Zustand, Django

Mobile / hybrid

React Native, Expo, Flutter, Android, Gradle

Testing / monitoring

Vitest, MSW, Storybook, Sentry, Ragas

Tools & Data

GitHub Actions, Jenkins, Docker, MariaDB, SQLite

Education & activities

Gyeongsang National University / Computer Science

Graduated 2023.02

GPA 3.64 / 4.5

Programmers Devcourse

2023.12 ~ 2024.05

Cloud Application Engineering, React and React Native

Programmers Devcourse / Assistant Mentor

2024.05 ~ 2024.09

Shared development resources and supported daily scrums and project reviews.

Awards

DIVE 2024 Global Data Hackathon

2024.10

Busan Technopark President Award / third in the challenge track

Gyeongnam Software Competition

2021.10

ESD HotDeal / Top Excellence Award

More

About
jukrap.vercel.app/en/about

Work
jukrap.vercel.app/en/work

Side Projects
jukrap.vercel.app/en/projects